금융 IT / 데이터 엔지니어 지원용 포트폴리오

김태형

TaeHyeong Kim

데이터가 빠지거나 틀리거나 두 번 처리되는 순간을 설계로 막아 왔습니다

경북대학교 컴퓨터학부 · 2027.02 졸업 예정 · xogud3624@gmail.com · github.com/KTHYEONG

증권사 API와 거래소 스트림에서 데이터를 모으고, 품질을 검증하고, 주문 직전까지 안전하게 연결하는 시스템을 혼자 만들었습니다. 장애가 났을 때 무엇이 빠졌는지 찾고 메우는 방법, 같은 요청을 두 번 보내지 않는 방법을 사례로 정리했습니다.

핵심 역량

수집 무결성

장중에는 파싱 없이 원본을 먼저 저장하고, 연결이 끊긴 구간은 순번으로 찾아 다른 증권사 API로 보충했습니다.

안전한 실행과 운영

주문을 보내기 전에 먼저 기록하고, 불확실하면 원장으로 확인해 중복 주문과 재실행 중복을 막았습니다.

데이터 정합성 검증

중복 집계로 부풀려진 합계를 멈추는 방식으로 막고, 검증 점수와 실제 점수의 차이는 원인을 나눠 확인했습니다.

대표 결과

SaveTax

약 24%

중복 주문 보정

중복을 그대로 더하면 부풀려지는 비율. 대회 데이터 안내 기준

코드로 확인

Port Entity Matching

0.8162 → 0.9477

스트레스 스위트 점수

20개 교란 시나리오, 로컬 모의 교란 기준

백테스트

Quant Dashboard

179개 통과

단위·계약 테스트

WSL에서 pytest 실행, 0.46초

코드로 확인

> **수치를 읽는 법**
>
> 모든 수치에는 측정 방식을 함께 표시했습니다. 실계좌 수익이나 대회 순위, 수상 실적(Emori 팀 논문 제외)은 이 자료에 포함하지 않았습니다. 자세한 근거와 실행 방법은 각 저장소 README에 있습니다.

프로젝트 한눈에 보기

01 KRX-Alpha 개인 진행 중

장중 틱을 놓치지 않고 모으고, 결손을 찾아 메우는 수집·검증 인프라

2026.09 ~ 진행 중 · 단독 설계·구현 (수집, 누락 감사·보충, 정규화, 원격 백업·보존, 모의 OMS, 구조 검사 테스트)

약 3만 줄 코드 규모

src 134개 파일 · 저장소 파일 집계

코드로 확인

약 2,860개 테스트

def test_ 개수 · 파라미터 확장 전

코드로 확인

github.com/KTHYEONG/krx-alpha

02 K-Closing Alpha 개인

주문 안전성(쓰기 전 기록)을 다룬 종가매매 자동 집행 시스템 (요약)

2026 · 기획부터 데이터 수집, 모델, 검증, 주문 실행까지 단독 설계·구현

+34.58bp 일평균 순수익

886거래일(2023-01~2026-09), 거래세·왕복 2틱 호가 비용·수수료 차감 후 (1bp=0.01%)

백테스트

2.73 Sharpe

t값 5.12, 승률 56.3%, 최대낙폭 28.98%

백테스트

github.com/KTHYEONG/k-closing-alpha

03 SaveTax 개인 완료

중복 집계와 허위 후보를 막는 공공 조달 데이터 분석 (해커톤)

2026.10 · 개인 프로젝트로, 설계와 구현을 단독으로 진행했습니다. AI 도구로 코드를 작성했고, 설계 방향과 비교 기준 결정, 수치와 동작 확인은 직접 했습니다.

약 24% 중복 주문 보정

중복을 그대로 더하면 부풀려지는 비율. 대회 데이터 안내 기준

코드로 확인

1,513개 단위 테스트

pytest 수집 기준, mypy strict와 ruff 통과

코드로 확인

비공개 저장소

04 Quant Dashboard 개인

서버 상태를 서버에 부담 없이 감시하는 대시보드 (보조)

2026.10 ~ · 단독 설계·구현 (감시 대상 수집기와 데이터는 별도 프로젝트)

179개 통과 단위·계약 테스트

WSL에서 pytest 실행, 0.46초

코드로 확인

0개 런타임 의존성

Python 표준 라이브러리만 사용 (pyproject.toml)

코드로 확인

비공개 저장소

05 **Port Entity Matching** 개인 진행 중

해운물류 데이터의 개체 매칭 대회 (진행 중)

2026.10 ~ 진행 중 · 개인 참가로 정규화, 후보 검색, 특징, 랭커, 평가 도구, 제출 코드를 단독 설계·구현했습니다.

0.9910 로컬 교차검증 점수

선박 단위 5-fold, 최종 채택 설정(LLM 채널 추가 전 번들)

백테스트

0.8162 → 0.9477 스트레스 스위트 점수

20개 교란 시나리오, 로컬 모의 교란 기준

백테스트

비공개 저장소

문제 해결 사례

CASE 01 KRX-Alpha 개인

개장 직후 메시지 폭주에도 놓치지 않도록, 장중에는 파싱하지 않고 원본만 기록했다

수신	원본 기록	L0 보관	장마감 후	L1 변환
WebSocket 메시지	recv_ts만 붙임	jsonl.zst 이어쓰기	파싱·중복 제거	Parquet + 품질 검사

P 문제

개장 직후에는 초당 메시지가 몰립니다. 수신 중에 파싱·검증까지 하면 처리가 밀려 메시지를 놓칠 수 있습니다.

A 분석

- 수신하며 바로 정제하는 방식은 구현이 단순하지만, 파싱 오류가 곧 데이터 손실이 되어 제외했습니다.
- 원본을 먼저 남기는 방식은 나중에 규칙을 고쳐도 다시 계산할 수 있어 선택했습니다.

A 실행

- 수신 경로는 받은 메시지에 수신 시각(recv_ts)만 붙여 압축 파일(.jsonl.zst)에 이어 쓰기만 하도록 했습니다.
- 파싱·중복 제거·품질 검사는 장마감 후 정규화 단계(L0 원본 → L1 Parquet)로 분리했습니다.

R 결과 코드로 확인

수신 경로와 해석 단계가 코드상 분리되어 있습니다. 실제 손실률은 측정해 기록하지 않았습니다.

CASE 02 KRX-Alpha 개인

연결이 끊긴 구간의 체결을 순번으로 찾아 다른 증권사 API로 보충했다

순번 감사	구간 확정	타사 보충	병합	출처 표시
종목별 빈틈 계산	시간대·종목 특정	키움·KIS 체결 이력	SQLite 외부 정렬	별도 파일로 저장

P 문제

수신 지연이 커지다 연결이 끊기면 그 구간의 체결은 증권사가 다시 보내 주지 않아 영구히 빠집니다.

A 분석

- 같은 증권사에 다시 요청해도 이미 지나간 체결은 받을 수 없어 제외했습니다.
- 다른 증권사의 체결 이력 API로 채우되, 어느 시간대·종목이 비었는지 먼저 정확히 알아야 한다고 판단했습니다.
- 원본을 고치지 않도록, 보충분은 출처(vendor, provenance)를 붙인 별도 파일로만 쌓았습니다.

A 실행

- 틱마다 붙은 체결 순번의 빈틈을 종목별로 계산하는 감사 단계를 만들었습니다.
- 키움증권 체결 이력과 한국투자증권 API로 빈 구간을 채우는 보충 단계를 만들었습니다.
- 원본과 보충분을 SQLite 외부 정렬로 합치며 중복을 제거하는 병합 단계를 만들었습니다.

R 결과 코드로 확인

감사 → 보충 → 병합이 각각 독립된 CLI 단계로 나뉘어 있고, L0 원본은 건드리지 않습니다. 증권사가 당일 체결만 제공하는 등 복구할 수 없는 구간은 남을 수 있습니다.

CASE 03 K-Closing Alpha 개인

주문을 보내기 전에 먼저 기록하고, 응답이 불확실하면 증권사 원장으로 확인해 중복 주문을 막았다

의도 기록	주문 전송	응답 불확실	원장 조회	채택/종결
INTENT 디스크 저장	SEND 기록	UNKNOWN 상태	같은 주문 탐색	ADOPT 또는 종결

P 문제

주문 응답이 타임아웃되면 접수 여부를 알 수 없습니다. 그대로 재전송하면 이중 매수가 됩니다.

A 분석

- 재시도 횟수를 제한하는 방법은 이미 접수된 주문을 막지 못해 제외했습니다.
- 보내기 전에 기록을 남기고, 결과가 불확실하면 증권사 원장을 조회해 맞추는 방식을 선택했습니다.

A 실행

- 주문 의도(INTENT)와 전송(SEND)을 디스크에 먼저 쓰고 전송했습니다.
- 응답이 불확실하면 UNKNOWN 상태로 두고, 원장에서 같은 주문을 찾으면 채택(ADOPT), 없을 때만 종결하도록 했습니다.
- 종가는 시각, 장운영 구분코드, 현재가와 호가 조회 가격 일치의 3조건을 모두 만족할 때만 확정합니다.

R 결과 코드로 확인

상태 전이와 채택 로직, 종가 확정 조건 함수를 구현했고 단위 테스트가 있습니다. 실거래 운영 기록은 없습니다.

CASE 04 SaveTax 개인

같은 주문이 여러 물품에 중복돼 합계가 부풀려지는 문제를, 불일치는 멈추는 방식으로 풀기

로드	무결성 검사	불일치 중단	정규화 뷰	집계
Parquet 직접 조회	dlv_key별 일치 확인	오류로 멈춤	주문당 1건	빈 값은 0 아님

P 문제

같은 주문 기록(dlv_key)이 여러 물품 아래에 반복돼, 단순 합계가 실제보다 큽니다. 집계 기준이 틀리면 이후의 모든 후보 판정이 어긋납니다.

A 분석

- 합계 전에 중복을 없애야 하지만, 묶은 기록끼리 계약번호나 금액이 다르면 합치면 안 됩니다.
- 주문이 없는 물품(약 71%)은 값이 비어 있는 것이지 0이 아니므로, 0으로 채우면 평균이 왜곡돼 제외했습니다.
- 불일치를 조용히 넘기는 것보다, 데이터를 불러오는 시점에 멈추는 쪽을 선택했습니다.

A 실행

- DuckDB로 Parquet을 직접 조회하며 dlv_key별 계약번호·금액이 하나로 일치하는지 검사합니다.
- 주문당 한 건으로 집계하는 정규화 뷰를 만들고, 불일치하면 오류로 중단합니다.
- 빈 값은 0으로 바꾸지 않고 그대로 둡니다.

R 결과 코드로 확인

주문당 한 건으로 집계하는 뷰를 쓰고, 불일치 데이터는 조용히 넘어가지 않고 중단됩니다. 약 24%라는 수치는 대회 데이터 안내의 값입니다.

CASE 05 Port Entity Matching 개인

교차검증 0.9875인데 리더보드는 0.8805였을 때, 원인을 가설로 나눠 하나씩 확인한 사례

P 문제

로컬 점수와 리더보드의 차이가 컸습니다. 원인을 모른 채 모델을 고치면 제출 기회만 낭비하므로 먼저 원인을 좁혀야 했습니다.

A 분석

- 학습과 서빙 경로 불일치는 저장한 모델과 교차검증 예측을 비교해 차이 0건이라 제외했습니다.
- 처음 보는 선박(정확도 0.998), 라틴 톤수 잡음(0.9965), 라틴 이름 오타(0.957)도 모두 낮은 점수를 설명하지 못해 제외했습니다.
- 토큰 순서를 바꾸거나 하나를 빼자 정확도가 0.4~0.5로 무너졌습니다. 학습 데이터는 한 선박이 늘 같은 철자를 써서 교차검증으로는 이 변화를 볼 수 없었습니다.

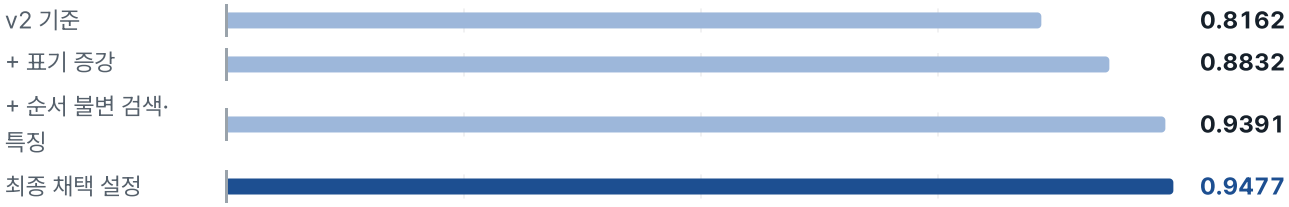
A 실행

- 깨끗한 교차검증 점수만으로 모델을 고르지 않고, 20개 교란 시나리오를 묶은 스트레스 스위트 점수를 함께 보기로 했습니다.
- 표기 증강을 학습에 넣고, 순서에 상관없는 검색(정렬 키, 토큰 색인)과 순서 불변 특징 7종을 추가했습니다.

R 결과 백테스트

로컬 스위트 점수가 0.8162에서 0.9477로 올랐고 토큰 순서를 바꾼 라틴 이름 정확도는 0.486에서 0.999가 됐습니다. 학습 데이터로 만든 모의 교란 기준이며 실제 테스트의 교란 비율을 보여 주지는 않습니다.

스트레스 스위트 점수 변화 점수 백테스트



선박 단위 5-fold, 모의 교란 20개 시나리오 (로컬)

추가 근거

사용한 기술

Python pytest PyArrow(Parquet) LightGBM systemd mypy ruff asyncio/WebSocket Polars SQLite zstandard Docker Compose pandas aiohttp pydantic DuckDB Parquet Decimal

개발 과정에서의 AI 활용

AI 코딩 도구를 주로 코드를 쓰는 데 사용했고, 아이디어를 다듬거나 처음 접하는 분야를 공부할 때도 썼습니다. 수치와 동작은 직접 확인했고, 확인하지 못한 것은 성과로 쓰지 않았습니다.

AI에 맡긴 일	직접 한 일
코드 작성	결과 수치와 동작을 직접 확인
아이디어 정리와 도메인 학습 보조	확인하지 못한 내용은 성과로 쓰지 않기

프로젝트별 한계

- **KRX-Alpha** 전략(알파) 단계가 없습니다. 신호 생성과 백테스트 엔진은 이 저장소에 없고, 수익 성과 수치도 없습니다.
- **K-Closing Alpha** 모든 수익 수치는 과거 데이터 백테스트이며, 실계좌나 모의투자의 누적 손익 기록은 없습니다. 118개 전략 중 좋은 결과를 골랐기 때문에 실제 성과는 이보다 낮을 수 있습니다.
- **SaveTax** 데이터(data/)와 참가자 토큰이 저장소에 없어, 저장소만으로는 CLI를 처음부터 끝까지 재현할 수 없습니다. 테스트는 합성 데이터로 돌아갑니다.
- **Quant Dashboard** 감시 대상의 데이터와 로그가 저장소에 없고 개인 서버 구조에 맞춰져 있어 다른 환경에서 재현하기 어렵습니다.
- **Port Entity Matching** 로컬 점수 0.9910은 LLM 채널을 넣기 전 번들의 값이고, LLM 채널을 포함한 현재 코드의 교차검증 점수는 아직 측정하지 않았습니다. 대회 진행 중이라 최종 순위도 없습니다.

수치 기준 표기

백테스트 과거 데이터 시뮬레이션 · 재현 시뮬레이션 체결·증거금 규칙을 적용한 원장 리플레이 · 평가셋 측정 정해진 평가 데이터로 잰 값
팀 측정 팀 전체 측정값(제 기여분만의 값이 아님) · 운영 기록 실제로 돌린 기록 · 코드로 확인 코드·설정·테스트로 확인한 설계 사실

그 밖의 프로젝트

auto-mobility 실내 공간을 3D 모델로 복원하고 센서 데이터로 검증하는 온라인 현장실습 팀 프로젝트(3명). 저는 Visual SLAM 비교와 평가를 주로 맡았습니다.

K-Stock Engine 국내주식 다중 팩터 전략을 연구하고 가상 장부로 기록합니다. 아직 불완전한 단계라 성과는 싣지 않았습니다.

OIBC 2026 태양광 발전량 예측과 가상발전소(VPP) 배정 대회 프로젝트. 제출 전이라 점수는 싣지 않았습니다.

연락 xogud3624@gmail.com · github.com/KTHYEONG